Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Handling edge cases before they bite

Today
Welcome to your next lesson: Handling edge cases before they bite.

I'm preparing your personalized lesson based on what you've learned, what's ahead, and the best way to help you build confidence.

Keep it up, every step counts.

Imagine you wrote this Python function:

Python

```
def divide(a, b):
    return a / b
```

You try:

Python

```
print(divide(10, 2))   # works
print(divide(10, 0))   # boom 
```

The first line is fine, the second crashes with “division by zero”. That crash is caused by an edge-case input.

In your own words, Ron: does “edge-case input” sound more like

the weird/rare values that break your function,
or
the normal values you expect most of the time?

You can just answer with a short phrase.

Weird or rare values

Exactly — edge cases are the weird or rare values that can break your function or make it behave strangely.

For inputs, we’ll use a simple checklist of 6 edge-case “buckets”:

Empty (like "" or [])
Zero (0)
Negative numbers
Extreme/big values
None
Wrong type (like "abc" when you expected a number)
Let’s focus on just one for now: wrong type.

Example:

Python

```
def divide(a, b):
    return a / b

print(divide("10", 2))   # type problem
```

This can blow up because "10" is a string, not a number.

Does the idea of “wrong type” as an edge case make sense so far, or should I show one more simple example before we add a fix?

It makes sense

Nice 👍

Now let’s see why guarding against edge cases is useful.

Take this function:

Python

```
def safe_divide(a, b):
    return a / b
```

Two inputs that can break it:

b = 0 → division by zero
a or b is the wrong type → like "10" or "abc"

We can protect the function by adding guard checks at the top, before the real work:

If types are wrong → return something safe
If b is zero → return something safe
Otherwise → do a / b (the normal “happy path”)

In steps, the shape is:

Python

```
def safe_divide(a, b):
    # 1) type check
    # 2) zero check
    # 3) normal division
```

Make sense, this “guards first, real work last” idea?

Yes, it makes sense

Great ☑

Let’s zoom in on one guard: blocking wrong types using isinstance.

Step by step:

Python

```
def safe_divide(a, b):
    if not isinstance(a, (int, float)):
        return None

    if not isinstance(b, (int, float)):
        return None
```

```
    return a / b
```

isinstance(a, (int, float)) checks: “Is a an int or a float?”
If it’s not, we return None early, before doing the division.

So the flow is:

Check types
Maybe return early
Only divide if both are numbers
Does this step-by-step use of isinstance feel clear, or should I restate it with a smaller example?

It's clear to me

What do you think the checks are doing here? Pick an option below 👇

Given this function:

```
def safe_divide(a, b):
    if not isinstance(a, (int, float)):
        return None
    if not isinstance(b, (int, float)):
        return None
    return a / b
```

What is the main purpose of the two if not isinstance(...) checks at the top?

A.

To make the code run faster

B.

To ensure a and b are numbers before dividing

C.

To automatically convert strings to numbers

D.

To make the function print an error message

☑ Correct!

Those two if not isinstance(...) lines are there to make sure a and b are numbers before dividing.

Step by step:

isinstance(a, (int, float)) → checks the type of a
if not ...: return None → if a (or b) is not a number, we exit early with a safe value instead of crashing on a / b.
Next tiny step: let’s look at another kind of edge case: zero as a divisor.

Quick question for you: if b is 0, what should safe_divide(a, b) do in your opinion—

return None, return 0, or return a message like "cannot divide by zero"?

not deivide and return None perhaps

Yeah, I like that choice 👍 — “don’t divide and return None” is a clean, simple rule.

Now let’s plug that into the function in a clear order:

Check types
Check for zero
Do the division

Like this:

Python

```
def safe_divide(a, b):
    if not isinstance(a, (int, float)):
        return None

    if not isinstance(b, (int, float)):
        return None

    if b == 0:
        return None

    return a / b
```

So any “bad” input (wrong type or zero) gets caught at the top and exits early. Only good inputs reach return a / b.

Make sense how the b == 0 check fits in with the earlier type checks?

yerah make sense ive coded with try: before too in practice seesions

Nice, that’s great context 👌

try:/except: is another way to handle problems, but in this lesson we’ll stick to preventing them with simple checks before they happen.

So far we’ve mainly talked about wrong type and zero as edge cases. I want to show you a simple “checklist” you can mentally run through when you design a function.

Here are the 6 common input edge-case buckets we’ll use:

Empty – "" (empty string), [] (empty list)
Zero – 0
Negative – like -5
Extreme/big – like 10_000_000
None – value is missing
Wrong type – "abc" instead of a number, etc.
Quick check, just by intuition:

if you had a function that calculates average speed = distance / time, which of these buckets do you think could be risky for the time input?

zero

Exactly 💯 — time = 0 is super dangerous there, because it leads to division by zero.

So for a function like:

Python

```
def average_speed(distance, time):
    # edge-case checks go here
    return distance / time
```

your checklist for time might be:

Is time the wrong type? (e.g. "10")
Is time None?
Is time zero? 👈 big one
Is time negative? (weird in most real-world cases)

Guard clauses would sit on top and block those before doing the math.

I'd like you to try a tiny guard-yourself exercise next:

Are you up for writing a small safe_divide in the code editor that:

returns None if a or b is not a number, or if b is 0
otherwise returns a / b?

If yes, I'll spell out the exact steps and you can implement it.

Yes, I'm ready

Awesome 😄 Let's do it step by step.

Your task in the code editor:

Create a function safe_divide(a, b) that:

uses isinstance to check that a and b are int or float
returns None if either is the wrong type
returns None if b is 0
otherwise returns a / b
After the function, add some print tests, for example:

a normal case: print(safe_divide(10, 2))
zero case: print(safe_divide(10, 0))
wrong type: print(safe_divide("10", 2))

Run it, then hit Submit code so I can see it and give feedback. 🚀

Python

```
def safe_divide(a, b):
    if not isinstance(a, (int, float)):
        return None

    elif not isinstance(b, (int, float)):
        return None
    elif b == 0:
        return None
    else:
        return a / b
print(safe_divide(10, 2))
print(safe_divide(10, 0))
print(safe_divide("10", 2))
```

Nice work, Ron 👏

Your function behaves exactly as we wanted:

safe_divide(10, 2) → 5.0 ☑
safe_divide(10, 0) → None ☑
safe_divide("10", 2) → None ☑

Your guard clauses are correct and in a clear order: type → zero → divide. Using elif here is fine since only one of those branches should run.

Next, let's practice spotting edge cases without coding.

Imagine this function:

Python

```
def miles_to_km(miles):
    return miles * 1.609
```

Which of these inputs for miles do you think are edge cases we should worry about?

Just list the ones you think are risky: 0, -5, 1000000, "10", None.

(negative distance nakes no sense), "10" (Strings not numberes, None (missing input), big numbes are fine id think and a 0 mile trip isnt an error perse

Love how you reasoned through each one 👌

Your picks are spot on:

-5 → ❌ negative distance is strange → edge case
"10" → ❌ wrong type (string) → edge case
None → ❌ missing value → edge case
0 → ☑ valid (0 miles is just “no distance”)
1000000 → ☑ allowed, though it’s an extreme value we might still keep in mind

So you’re already using the checklist: negative, wrong type, None, extreme.

Next step: want to try adding guards to a miles_to_km function, similar to what you did with safe_divide, but for negative numbers and wrong types?

Python

```
def miles_to_km(miles):

    if not isinstance(miles, (int, float)):
        return None

    elif miles < 0:
        return None
    elif miles == None:
        return None
    else:
        return miles * 1.609

print(miles_to_km(-5))
print(miles_to_km(""))
print(miles_to_km(None))
print(miles_to_km(1000))
print(miles_to_km(0))
print(miles_to_km(40000000000000))
```

Nice job, Ron — this is solid 👍

Your miles_to_km:

Python

```
def miles_to_km(miles):
    if not isinstance(miles, (int, float)):
        return None
    elif miles < 0:
        return None
    elif miles == None:
        return None
    else:
        return miles * 1.609
```

Behavior from your tests:

-5 → None ☑ (negative blocked)
"" → None ☑ (wrong type blocked by isinstance)
None → None ☑ (caught by isinstance too)
1000, 0, 40000000000000 → valid numbers → converted ☑

Small note: the miles == None check is never reached, because None already fails the isinstance(miles, (int, float)) check. So you could safely delete that elif and the function would behave the same.

You're already:

spotting edge cases ☑
writing guard clauses with isinstance and comparisons ☑

Want to do a quick multiple□choice check on edge-case categories to lock this in?

good to know

You just found a real edge-case redundancy there 😄

Let's do that quick multiple□choice check to lock things in.

Look at this function:

Python

```
def describe_length(text):
    if text is None:
        return None
    if not isinstance(text, str):
        return None
    if text == "":
        return "empty"
    return f"length is {len(text)}"
```

What kind of edge case is the text == "" check mainly guarding against?

A) Zero
B) Empty
C) Wrong type
D) None

Just answer with the letter.

b empty

☑ Exactly — B) Empty is right.

Step by step:

text is None → guarding the None edge case
not isinstance(text, str) → guarding wrong type
text == "" → guarding the empty string case specifically

You've now:

identified edge cases from examples
written guard clauses with isinstance
handled 0, negative, None, wrong type, and empty 🎯
Two quick questions for you:

Do you feel comfortable spotting edge cases using this mental checklist: empty, zero,

negative, extreme, None, wrong-type?

Are you ready to wrap up this edge□case lesson and move on to the next one, or want one more practice question first?

i feel comforatable yeah and i agree its nice to do things yoiuself in code and not alawys rely on systm livs like try and yeah im ready to move on

Love that mindset, Ron — building your own guards instead of always leaning on try/except is a real developer habit 💪

Quick recap of what you nailed in this lesson:

You can spot edge cases using the 6-bucket checklist: empty, zero, negative, extreme, None, wrong type.

You wrote safe_divide and miles_to_km with guard clauses that:

use isinstance to validate types

block bad values (0, negatives, wrong types, None)

keep the "happy path" (real work) at the bottom

This is exactly how you avoid bugs before they bite in real projects.

You're all set to move on to the next lesson 🎓

When you're ready, hit Next in the Learn tab and we'll keep leveling you up.